

FINAL TECHNICAL REPORT

Dokumen Kompilasi Implementasi dan User Manual

Bacarita

Platform Pembelajaran Membaca Adaptif Berbasis Web
Terintegrasi AI Eye-Tracking untuk Anak Disleksia

Repositori

KOM1315_SmtGenap26_Kelompok02_Bacarita

Program

Mata Kuliah Keamanan Informasi (KOM1315)

Versi Dokumen

Final Release - 10 Juni 2026

Daftar Isi

Daftar Isi

1. Ringkasan Proyek	3
1.1. Latar Belakang	3
1.2. Tujuan Final Release	3
1.3. Ruang Lingkup Implementasi	3
2. Struktur Implementasi	3
2.1. Struktur Repositori	3
2.2. Komponen Backend	4
2.3. Komponen Frontend	4
2.4. Logging Dashboard	4
3. Implementasi Keamanan	5
3.1. Authentication	5
3.2. Authorization	5
3.3. Accounting dan Audit Log	5
3.4. Digital Signature	6
3.5. Enkripsi Data Sensitif	6
3.6. Keamanan Database	7
4. Alur Utama Aplikasi	7
4.1. Registrasi dan Manajemen Akun	7
4.2. Login Multi-Peran	7
4.3. Sesi Membaca dan Eye-Tracking	7
4.4. Kurasi dan Persetujuan Cerita	7
5. Hasil Pengujian	7
5.1. Bukti Pengujian Keamanan	7
5.2. Ringkasan Hasil	8
5.3. Keterbatasan yang Masih Perlu Ditingkatkan	8
6. Panduan Instalasi dan Operasional	8
6.1. Prasyarat	8
6.2. Menjalankan Backend	8
6.3. Menjalankan Frontend	9
6.4. Menjalankan Logging Dashboard	9
6.5. Variabel Lingkungan Penting	9
7. Panduan Pengguna	9
7.1. Siswa	9
7.2. Guru	9
7.3. Orang Tua	10
7.4. Admin dan Kurator	10
8. Kesimpulan	10

1. Ringkasan Proyek

1.1. Latar Belakang

Bacarita adalah platform pembelajaran membaca berbasis web yang dirancang untuk membantu anak disleksia melalui pendekatan multimodal. Implementasi saat ini menggabungkan *eye-tracking* berbasis kamera di sisi klien, evaluasi pembacaan lisan berbasis sesi tes, kurikulum cerita berlevel, integrasi AI melalui OpenRouter untuk pembuatan soal STT, serta dashboard pemantauan bagi guru, orang tua, dan admin. Pada fase final ini, fokus pengembangan tidak hanya pada fungsionalitas aplikasi, tetapi juga pada penguatan kontrol keamanan yang sesuai dengan kebutuhan data anak dan aktivitas pembelajaran.

1.2. Tujuan Final Release

Dokumen ini menyatukan hasil implementasi teknis, struktur kode sumber, kontrol keamanan, pengujian, dan panduan operasional sistem. Dengan demikian, laporan ini berfungsi sebagai:

- kompilasi artefak teknis untuk penilaian akhir;
- jembatan antara dokumen desain awal dan implementasi aktual;
- panduan instalasi, pengoperasian, dan verifikasi fitur inti;
- ringkasan penerapan kontrol AAA, digital signature, dan perlindungan database.

1.3. Ruang Lingkup Implementasi

Final release Bacarita mencakup empat area utama yang saling terintegrasi:

- aplikasi backend berbasis NestJS untuk autentikasi, otorisasi, logika domain, dan manajemen data;
- aplikasi frontend berbasis Next.js untuk pengalaman belajar dan dashboard per peran;
- halaman audit login di aplikasi frontend admin untuk meninjau *security audit log* langsung dari backend;
- dashboard logging terpisah untuk visualisasi *security audit log* melalui *standalone proxy*.

2. Struktur Implementasi

2.1. Struktur Repositori

Struktur repositori akhir disusun agar selaras dengan arahan pengelolaan PBL, namun tetap mempertahankan organisasi kode yang stabil selama pengembangan.

Folder	Keterangan
01_Proposal_&_Analisis/	Proposal teknis dan dokumen threat modeling awal.
02_Design_Documents/	Dokumen ERD, diagram arsitektur, dan rencana pengujian AAA.
03_Source_Code/backend/	Implementasi utama server, domain logic, autentikasi, database, dan logging.
03_Source_Code/frontend/	Aplikasi web untuk siswa, guru, orang tua, admin, dan kurator.
03_Source_Code/logging-dashboard/	Dashboard visual untuk aktivitas audit dan hasil uji keamanan.
03_Source_Code/database/	Entrypoint kompatibilitas untuk artefak database yang direferensikan dari backend.

Folder	Keterangan
03_Source_Code/ digital_signature/	Entrypoint kompatibilitas untuk modul tanda tangan digital RSA.
04_Reports_&_Paper/	Laporan monitoring, laporan teknis final, dan tempat paper ilmiah.
05_Testing/	Log pengujian keamanan dan audit aktivitas AAA.

2.2. Komponen Backend

Backend Bacarita dibangun dengan NestJS 11 dan TypeScript. Struktur modul utamanya dipisahkan ke dalam beberapa area:

- `src/feature/auth/` untuk login multi-peran, AuthGuard, audit log autentikasi, dan digital signature RSA;
- `src/feature/users/` untuk domain akun guru, siswa, orang tua, admin, dan kurator;
- `src/feature/account-management/` untuk pendaftaran siswa oleh guru beserta relasi orang tua;
- `src/feature/dashboard/` untuk ringkasan dan detail dashboard guru serta orang tua;
- `src/feature/levels/` untuk konten cerita berlevel dan progres belajar;
- `src/feature/story-management/` untuk manajemen cerita oleh admin dan persetujuan cerita oleh kurator;
- `src/feature/test-session/` untuk sesi evaluasi membaca, STT, *eye distraction event*, ringkasan distraksi, skor, medal, dan tanda tangan hasil;
- `src/feature/ai/` untuk integrasi OpenRouter pada pembangkitan soal pre-test dan soal dari cerita;
- `src/feature/logging/` untuk audit log autentikasi (`auth_audit_logs`), `ApiAuditInterceptor` global, dan endpoint dashboard audit pada `auth/admin`;
- `src/common/crypto/` untuk enkripsi AES-256-CBC (*value transformer* TypeORM) dan hash token SHA-256;
- `src/database/`, `src/config/database/`, dan `src/migrations/` untuk *seed*, bootstrap, dan perubahan skema.

2.3. Komponen Frontend

Frontend utama menggunakan Next.js 15, React 19, dan Tailwind CSS v4. Antarmuka dipisahkan per peran pengguna agar jalur kerja lebih jelas, dengan route untuk siswa, guru, orang tua, admin, dan kurator. Di dalam frontend ini juga terdapat *session relay route* untuk menyimpan token dan role sebagai *HttpOnly cookie*, middleware proteksi route per peran, halaman admin untuk audit login, serta komponen *eye-tracking* dan *focus detection* yang dijalankan di sisi klien.

2.4. Logging Dashboard

Dashboard logging dibangun sebagai aplikasi Next.js terpisah pada folder `03_Source_Code/logging-dashboard/` (Next.js 16, React 19). Akses dashboard kini diamankan melalui autentikasi admin: admin login pada halaman `/login`, lalu route `/api/auth/login` meneruskan kredensial ke backend `auth/admins/login` dan menyimpan JWT sebagai *HttpOnly cookie* `auth_token`. Sebuah *middleware proxy.ts* menjaga seluruh halaman dan memproksikan dua route data, yaitu `/api/audit-logs` ke `auth/admin/audit-logs/standalone` dan `/api/api-logs` ke `auth/admin/api-logs`, sambil melampirkan header `Authorization: Bearer` dari cookie. Dashboard menyajikan dua halaman, yaitu monitor log autentikasi (*auth logs*) dan monitor permintaan API (*API logs*). Pendekatan berbasis JWT ini menggantikan mekanisme `x-audit-dashboard-key` sebelumnya sehingga konsisten dengan

kontrol otorisasi backend, sekaligus mendemonstrasikan komponen *Accounting* dari AAA secara eksplisit.

3. Implementasi Keamanan

3.1. Authentication

Lapisan autentikasi mendukung lima peran: teacher, student, parent, admin, dan curator. Implementasi final mencakup:

- JWT sebagai *access token*;
- penyimpanan token dan role dalam *HttpOnly cookie* melalui route frontend `/api/auth/set-session`;
- pembatasan laju login berbasis *ThrottlerGuard* dengan batas global 5 request / 60 detik;
- *account lockout* setelah 5 kegagalan login selama 15 menit;
- validasi panjang minimum password pada DTO login/registrasi sesuai peran (6 atau 8 karakter, tergantung endpoint);
- verifikasi hash token SHA-256 terhadap nilai di database.

Kontrol ini ditujukan untuk menutup temuan awal pada *threat modeling*, khususnya risiko pencurian token, *credential stuffing*, dan penyalahgunaan sesi.

3.2. Authorization

Otorisasi diterapkan melalui kontrol akses berbasis peran (RBAC) dengan enam peran pada enumerasi *AuthRole*, yaitu ADMIN, CURATOR, TEACHER, STUDENT, PARENT, dan ANY. Pembatasan akses dideklarasikan secara deklaratif menggunakan dekorator `@Auth()` pada setiap endpoint, misalnya `@Auth(AuthRole.ADMIN)` untuk endpoint audit log. Pendekatan ini memisahkan jalur akses berdasarkan peran operasional masing-masing pengguna:

- siswa hanya mengakses alur belajar dan tes membaca;
- guru mengelola murid serta memantau performa;
- orang tua melihat perkembangan anak;
- admin mengelola konten tingkat sistem dan seluruh audit log;
- kurator meninjau dan menyetujui cerita.

Penegakan otorisasi terjadi pada *AuthGuard* yang menjalankan tiga lapis validasi berurutan untuk setiap permintaan terproteksi: (1) verifikasi tanda tangan JWT terhadap `JWT_SECRET`, gagal menghasilkan 401; (2) pencocokan hash SHA-256 token dengan nilai di database untuk memastikan token belum dicabut, token tercabut ditolak 401; dan (3) perbandingan peran token dengan peran yang disyaratkan dekorator, ketidaksesuaian menghasilkan 403. Hanya permintaan yang lolos ketiga lapis yang diteruskan ke *controller*. Mekanisme ini mengurangi risiko *privilege escalation* dan pencurian token sesi sekaligus memastikan setiap pengguna hanya mengakses data yang relevan.

3.3. Accounting dan Audit Log

Komponen *Accounting* diwujudkan melalui log keamanan terstruktur untuk peristiwa seperti:

- login berhasil;
- login gagal;
- akun terkunci;
- logout.

Pada implementasi saat ini, event di atas disimpan ke tabel `auth_audit_logs` di database lengkap dengan `userId`, `role`, `ipAddress`, `userAgent`, dan `createdAt`, serta ditulis sebagai baris JSON ke berkas `auth_activity.log` sebagai bukti sekunder. Selain audit autentikasi, sistem juga mencatat seluruh permintaan API melalui `ApiAuditInterceptor`, sebuah *interceptor* global yang membungkus setiap *controller* dan merekam `method`, `endpoint`, `statusCode`, `durationMs`, `ipAddress`, serta `userId` ke tabel `api_audit_logs`.

Kedua sumber audit ini dikonsumsi oleh dashboard logging melalui endpoint `auth/admin/audit-logs/standalone` dan `auth/admin/api-logs` yang diproteksi `@Auth(AuthRole.ADMIN)`. Dashboard menyajikan ringkasan jumlah login berhasil, gagal, penguncian, dan logout, tren aktivitas per rentang waktu (24 jam, 7 hari, 30 hari, 90 hari), serta tabel rinci dengan penyaringan dan paginasi.

3.4. Digital Signature

Implementasi tanda tangan digital menggunakan RSA-SHA256. Modul ini menghasilkan pasangan kunci dari variabel lingkungan bila tersedia, atau membuat pasangan ephemeris untuk lingkungan pengembangan. Pada kode saat ini, tanda tangan diterapkan ketika `finishTestSession()` menyimpan hasil akhir sesi tes, dengan payload yang mencakup `id`, `studentId`, `score`, dan `medal`, lalu disimpan ke `resultSignature`. Hal ini memberikan jejak integritas dan *non-repudiation* pada hasil evaluasi siswa.

Komponen	Implementasi	Tujuan Keamanan
Algoritma	RSA-SHA256	Menjamin integritas dan non-repudiation
Private key	Dapat disuplai melalui environment	Menghindari hardcode kunci di repositori
Public key	Dapat diekspos untuk verifikasi	Memungkinkan verifikasi hasil secara terpisah
Integrasi	Finish test session dan unit test signature	Membuktikan modul dipakai di alur hasil evaluasi

3.5. Enkripsi Data Sensitif

Selain hashing kata sandi dan token, data identitas pribadi (PII) dienkripsi saat *at-rest* menggunakan AES-256-CBC. Enkripsi diterapkan secara transparan melalui `ValueTransformer` TypeORM sehingga data terenkripsi saat ditulis dan terdekripsi saat dibaca tanpa mengubah logika domain. Setiap operasi menggunakan *initialization vector* (IV) acak 16 byte yang disisipkan di awal *ciphertext* dengan format `ivHex:ciphertextHex`, sehingga dua *plaintext* identik menghasilkan *ciphertext* berbeda. Kunci 256-bit dipasok melalui `AES_SECRET_KEY` (heksadesimal 64 karakter).

Tabel	Kolom	Alasan
<code>students</code>	<code>fullName</code>	PII anak (minor)
<code>parents</code>	<code>fullName</code>	PII orang tua
<code>teachers</code>	<code>fullName</code> , <code>schoolName</code>	PII guru dan data institusi
<code>admins</code>	<code>fullName</code>	PII admin
<code>curators</code>	<code>fullName</code> , <code>phoneNumber</code>	PII kurator
<code>test_sessions</code>	<code>passageAtTaken</code> , <code>descriptionAtTaken</code>	Konten asesmen siswa

3.6. Keamanan Database

Lapisan database menggunakan MySQL dan TypeORM. Penguatan yang dilakukan pada tahap akhir meliputi:

- migrasi untuk kolom keamanan tambahan;
- penyimpanan hash token, bukan token mentah;
- penambahan kolom *failed login attempts* dan *locked until*;
- pengelolaan skema melalui migrasi terversi;
- seeder terpisah untuk data awal yang konsisten.

Pendekatan ini meningkatkan keterlacakan perubahan skema sekaligus mempermudah verifikasi kontrol keamanan terhadap data persisten.

4. Alur Utama Aplikasi

4.1. Registrasi dan Manajemen Akun

Guru dapat melakukan registrasi akun guru serta menambahkan siswa dan relasi orang tua melalui backend. Seeder juga menyiapkan akun admin dan kurator berbasis environment untuk keperluan pengelolaan konten. Aturan validasi password saat ini tidak sepenuhnya seragam antar endpoint, namun semua alur login dan registrasi tetap melewati validasi DTO dan hashing bcrypt.

4.2. Login Multi-Peran

Pengguna melakukan login melalui antarmuka sesuai peran. Setelah kredensial diverifikasi, token ditetapkan melalui jalur session yang lebih aman. Pada setiap request berikutnya, backend memvalidasi token dan memeriksa kesesuaian peran sebelum memberikan akses data.

4.3. Sesi Membaca dan Eye-Tracking

Siswa membaca materi sesuai level. Komponen *eye-tracking* dan *focus detection* dijalankan di browser untuk mengamati pola perhatian, sementara backend mengelola sesi tes, soal STT, penyimpanan jawaban, event distraksi, ringkasan distraksi, perhitungan skor, dan medal. Untuk pre-test dan pembuatan soal dari cerita, backend memanfaatkan OpenRouter agar prompt evaluasi dapat dibangkitkan dinamis.

4.4. Kurasi dan Persetujuan Cerita

Cerita baru atau perubahan konten melewati jalur admin dan kurator. Dengan demikian, konten pembelajaran tidak langsung masuk ke alur utama tanpa proses validasi.

5. Hasil Pengujian

5.1. Bukti Pengujian Keamanan

Pengujian keamanan yang terdokumentasi langsung di repositori saat ini terdiri dari:

- verifikasi AuthGuard untuk kasus akses benar, salah peran, dan tanpa token;
- verifikasi modul RSA signature terhadap payload valid, payload termodifikasi, dan signature palsu;
- pengujian unit untuk komponen inti backend lain yang terkait alur domain;
- *source code* e2e test untuk autentikasi admin, guru, siswa, orang tua, dan dashboard.

Artefak eksekusi yang dibundel dalam repositori tersedia pada `05_Testing/unit-test-security.log`, sedangkan `05_Testing/auth_activity.log` hanya berisi contoh inisialisasi log, bukan dump lengkap aktivitas runtime.

5.2. Ringkasan Hasil

Area Uji	Status	Catatan
AuthGuard AAA	PASS	Log unit test menunjukkan validasi 401 dan 403 berjalan sesuai ekspektasi.
Digital Signature RSA	PASS	Log unit test menunjukkan payload valid lolos, payload dan signature termodifikasi ditolak.
Regression Unit Backend	PASS	05_Testing/unit-test-security.log mencatat 9 suite dan 59 test lulus.
E2E Auth dan Dashboard	AVAILABLE	Spec tersedia di backend/test/e2e, namun log eksekusinya tidak dibundel pada repositori.
Security Audit Log	IMPLEMENTED	Audit disimpan di database dan divisualisasikan di frontend admin maupun dashboard standalone.

5.3. Keterbatasan yang Masih Perlu Ditingkatkan

Walaupun kontrol inti telah terpasang, beberapa perbaikan lanjutan masih layak dipertimbangkan:

- rotasi kunci RSA secara terjadwal untuk lingkungan produksi;
- normalisasi path file audit agar tidak bergantung pada path absolut lokal pengembang;
- enkripsi lebih lanjut untuk data kognitif yang sangat sensitif;
- *end-to-end testing* lintas frontend-backend yang lebih luas;
- penguatan fallback bila layanan OpenRouter tidak tersedia;
- integrasi observabilitas produksi untuk audit log real-time.

6. Panduan Instalasi dan Operasional

6.1. Prasyarat

Lingkungan minimum yang diperlukan:

- Node.js 18 atau lebih baru;
- MySQL;
- npm;
- konfigurasi variabel lingkungan backend, frontend, dan dashboard standalone.

6.2. Menjalankan Backend

1. Buka folder 03_Source_Code/backend/.
2. Jalankan `npm install`.
3. Salin `.env.example` menjadi `.env.development` untuk pengembangan lokal.
4. Buat folder `logs/` bila dibutuhkan oleh logger lokal.
5. Jalankan migrasi dengan `npm run migration:run`.
6. Jalankan seed awal dengan `npm run db:seed`.
7. Mulai server pengembangan dengan `npm run start:dev`.

Catatan: backend tidak membaca file `.env` biasa. Aplikasi memuat `.env.development`, `.env.production`, atau `.env.test` sesuai `NODE_ENV`.

6.3. Menjalankan Frontend

1. Buka folder 03_Source_Code/frontend/.
2. Jalankan `npm install`.
3. Siapkan `.env.local` dengan `NEXT_PUBLIC_API_URL` yang mengarah ke backend.
4. Jalankan `npm run dev`.

6.4. Menjalankan Logging Dashboard

1. Buka folder 03_Source_Code/logging-dashboard/.
2. Jalankan `npm install`.
3. Siapkan `.env.local` dengan `BACARITA_API_URL` yang mengarah ke backend.
4. Jalankan `npm run dev`.
5. Bila frontend utama juga berjalan di port default Next.js, jalankan salah satunya pada port berbeda.
6. Login menggunakan akun admin, lalu gunakan dashboard untuk demonstrasi log audit, *alert*, dan ringkasan hasil uji.

6.5. Variabel Lingkungan Penting

Variabel	Fungsi
DB_HOST, DB_PORT, DB_USERNAME, DB_PASSWORD, DB_DATABASE	Koneksi database utama backend.
JWT_SECRET, JWT_EXPIRES	Konfigurasi token autentikasi backend.
AES_SECRET_KEY	Kunci 256-bit (heksadesimal 64 karakter) untuk enkripsi AES-256-CBC data PII.
MAIL_HOST, MAIL_PORT, MAIL_USER, MAIL_PASS	Pengiriman notifikasi email.
RSA_PRIVATE_KEY, RSA_PUBLIC_KEY	Kunci untuk tanda tangan digital produksi.
OPENROUTER_API_KEY, OPENROUTER_BASE_URL, OPENROUTER_MODEL	Akses integrasi AI/LLM untuk pembangkitan soal STT.
NEXT_PUBLIC_API_URL	Base URL backend untuk frontend utama.
BACARITA_API_URL	Base URL backend untuk logging dashboard standalone.

7. Panduan Pengguna

7.1. Siswa

Siswa masuk melalui halaman login siswa, menjalani pre-test atau level yang tersedia, membaca cerita, dan mengikuti sesi evaluasi. Sistem kemudian menyajikan progres berdasarkan hasil yang terkumpul.

7.2. Guru

Guru menggunakan dashboard untuk:

- melihat performa murid;
- menambahkan murid baru;
- memonitor hasil pembacaan;
- menggunakan data performa sebagai dasar intervensi belajar.

7.3. Orang Tua

Orang tua dapat memantau perkembangan anak melalui dashboard yang lebih ringkas, dengan fokus pada capaian belajar dan performa membaca.

7.4. Admin dan Kurator

Admin mengelola level, cerita, dan audit login melalui dashboard admin, sedangkan kurator memeriksa cerita berstatus menunggu persetujuan, melihat histori approval, lalu menyetujui atau menolak cerita. Pemisahan tanggung jawab ini membantu menjaga kualitas konten sekaligus memperkuat kontrol akses.

8. Kesimpulan

Bacarita telah berkembang dari artefak desain awal menjadi sistem yang memiliki implementasi backend, frontend, dashboard audit, serta kontrol keamanan yang terintegrasi. Repositori akhir tidak hanya memuat source code, tetapi juga dokumentasi, log pengujian, migrasi database, dan modul tanda tangan digital yang relevan dengan kebutuhan mata kuliah Keamanan Informasi.

Secara khusus, final release ini menunjukkan bahwa kontrol AAA, hash token database, *rate limiting*, *account lockout*, audit logging berbasis database, integrasi OpenRouter, dan RSA digital signature telah masuk ke alur utama aplikasi. Dengan revisi laporan ini, deskripsi teknis kini lebih selaras dengan struktur folder, modul aktif, variabel lingkungan, dan bukti pengujian yang benar-benar tersedia pada repositori saat ini.